

Controlador de Teclado Matricial con Banco de Registros en FPGA para la Ejecución de Comandos AT en Sistemas Operativos

Renzo Felipe Chuzon Wickham¹, Jorge David Rivera Flores¹, Andrea Alexandra Rodriguez Vasquez¹

¹Universidad Nacional Mayor de San Marcos, Facultad de Ingeniería Electrónica y Eléctrica, Lima, Perú.

Resumen—El presente artículo detalla el diseño e implementación de un controlador de teclado matricial con banco de registros en FPGA para la ejecución de comandos AT en sistemas operativos. La arquitectura de hardware propuesta se estructura en tres capas orientadas a la interacción física determinista con interfaces web y entornos de escritorio. El núcleo lógico destaca por una Máquina de Estados Finitos (FSM) que gestiona el escaneo secuencial a través de una base de tiempos dedicada y un decodificador combinatorial puro que transforma las coordenadas matriciales en comandos binarios con un retardo de propagación mínimo en el orden de los nanosegundos. Asimismo, se incorpora un banco de registros encargado del almacenamiento temporal y formateo de los datos para su transmisión serial asíncrona hacia un puente de enrutamiento. Este enfoque de partición garantiza la captura de datos con cero jitter mecánico, proveyendo una solución robusta, de baja latencia y alta confiabilidad para entornos de control interactivos.

Palabras clave: FPGA, Verilog, Controlador Matricial, Banco de Registros, Comandos AT, Máquina de Estados Finitos (FSM).

Abstract—This paper details the design and implementation of a matrix keypad controller with a register bank in an FPGA for AT command execution in operating systems. The proposed hardware architecture is structured in three layers aimed at deterministic physical interaction with web interfaces and desktop environments. The logical core stands out due to a Finite State Machine (FSM) that manages sequential scanning through a dedicated time base and a pure combinatorial decoder that transforms matrix coordinates into binary commands with minimal propagation delay in the nanosecond range. Furthermore, a register bank is incorporated for the temporary storage and formatting of data for asynchronous serial transmission toward a routing bridge. This partitioning approach guarantees data capture with zero mechanical jitter, providing a robust, low-latency, and highly reliable solution for interactive control environments.

Keywords: FPGA, Verilog, Keypad Controller, Register Bank, AT Commands, Finite State Machine (FSM).

I. INTRODUCCIÓN

El desarrollo de Interfaces Humano-Máquina (HMI) confiables requiere infraestructuras capaces de procesar estímulos electromecánicos físicos con alta precisión antes de traducirlos en acciones complejas dentro de ecosistemas de

software asíncronos. Tradicionalmente, la implementación exclusiva mediante software introduce vulnerabilidades críticas: latencia variable (jitter) y falsos positivos derivados del rebote mecánico [4] inherente a los conmutadores físicos de un teclado matricial.

El presente proyecto justifica la delegación de las tareas de adquisición y filtrado a una capa de hardware digital dedicada, sintetizada en una FPGA. Al procesar las señales mecánicas mediante hardware concurrente, se asegura un determinismo temporal estricto que el software de alto nivel no puede garantizar. El objetivo principal es diseñar un nodo embebido (FPGA MAX 10) [1] que capture eficazmente las entradas de un teclado 4x4, las procese mediante un decodificador lógico combinatorial puro y delegue la comunicación a un bus serial, liberando así los recursos del procesador host y asegurando una interacción ininterrumpida con un entorno web asíncrono.



Fig 1. Prototipo físico del sistema híbrido implementado. Se observa la interconexión entre la FPGA MAX 10 [1], el Arduino UNO [3] y la computadora host ejecutando el entorno web en React.

I. DISEÑO Y ARQUITECTURA DEL HARDWARE

La capa física del sistema fue descrita en lenguaje Verilog [2] y sintetizada en la FPGA. Su arquitectura se diseñó bajo un enfoque modular, priorizando el acondicionamiento de la señal mecánica y su decodificación inmediata.

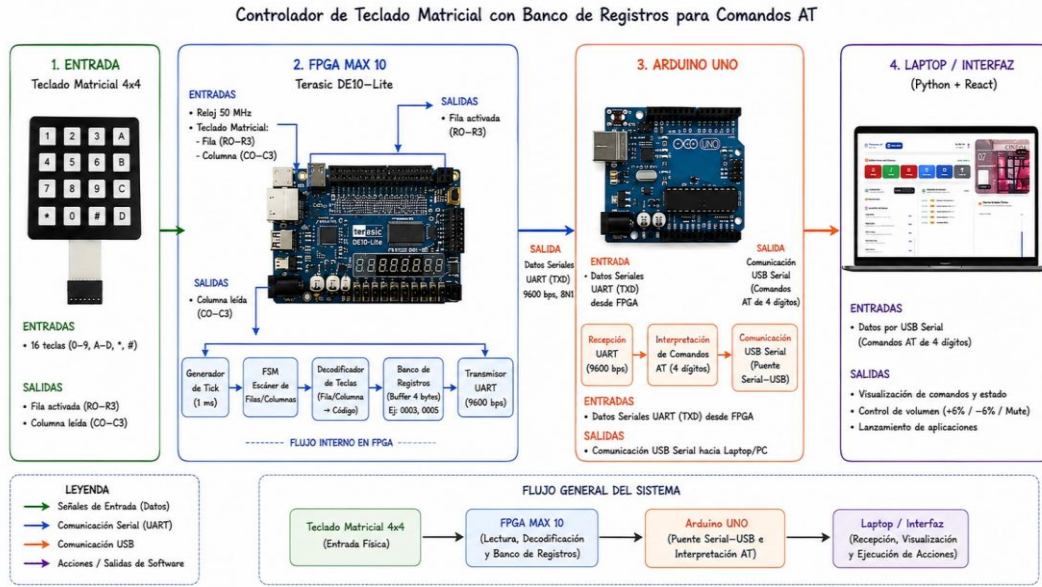


Fig 2. Diagrama de bloques de la arquitectura del sistema. Se detalla el flujo de datos multicapa: entrada física (teclado matricial), procesamiento lógico y almacenamiento (FPGA), enrutamiento asíncrono (Arduino) y capa de aplicación (Laptop).

A. Base de Tiempos y Máquina de Estados (FSM) de Escaneo

Para la correcta adquisición de datos del teclado matricial, se implementó una base de tiempos en hardware basada en el reloj maestro de la FPGA (50 MHz). Este divisor de frecuencia genera un pulso de habilitación (Clock Enable) llamado `tick_1ms`, el cual dicta el ritmo de las transiciones del sistema.

Este pulso de 1 milisegundo gobierna la Máquina de Estados Finitos (FSM) diseñada para el escaneo secuencial de las filas del teclado. Como se evidencia en la síntesis lógica [5], la FSM está compuesta por cuatro estados principales: S0_FILA0, S1_FILA1, S2_FILA2 y S3_FILA3.

Durante cada estado, el sistema asigna un nivel lógico bajo (0 lógico) a una fila específica, desplazándolo secuencialmente. Esta conmutación cíclica controlada por el `tick_1ms` garantiza un escaneo estable y permite que la circuitería combinacional posterior evalúe las columnas correspondientes de manera sincronizada y sin solapamiento de señales.

```
// =====
// 2. MÁQUINA DE ESTADOS (FSM) PARA ESCANEO DE FILAS
// =====
localparam S0_FILA0 = 2'b00;
localparam S1_FILA1 = 2'b01;
localparam S2_FILA2 = 2'b10;
localparam S3_FILA3 = 2'b11;

reg [1:0] estado_actual, estado_siguiente;

always @(posedge clk) begin
    if (~reset) begin
        estado_actual <= S0_FILA0;
    end else if (tick_1ms) begin // <--- CLOCK ENABLE
        estado_actual <= estado_siguiente;
    end
end

always @(*) begin
    case (estado_actual)
        S0_FILA0: begin row = 4'b1110; estado_siguiente = S1_FILA1; end
        S1_FILA1: begin row = 4'b1101; estado_siguiente = S2_FILA2; end
        S2_FILA2: begin row = 4'b1011; estado_siguiente = S3_FILA3; end
        S3_FILA3: begin row = 4'b0111; estado_siguiente = S0_FILA0; end
        default: begin row = 4'b1111; estado_siguiente = S0_FILA0; end
    endcase
end
```

Fig 3. Fragmento del código Verilog [2] evidenciando la Máquina de Estados Finitos (FSM) empleada para el escaneo de las filas del teclado. Se observan los estados secuenciales (S0_FILA0 a S3_FILA3) controlados por un pulso de habilitación de 1 ms.

B. Decodificador Combinacional Matricial

Una vez que la Máquina de Estados Finitos (FSM) conmuta cíclicamente las filas del teclado aplicando un cero lógico (0 activo bajo) mediante los vectores 4'b1110, 4'b1101, 4'b1011 y 4'b0111, las señales son enviadas al bloque de decodificación. Este componente constituye un punto crítico del diseño de hardware, ya que actúa como el traductor directo entre las coordenadas físicas bidimensionales del teclado y la lógica binaria que el sistema requiere para estructurar los comandos.

El módulo fue implementado como un Decodificador Combinacional Puro utilizando asignaciones concurrentes en Verilog [2]. Su comportamiento lógico se basa en la evaluación inmediata de la intersección entre la fila activada por la FSM ($R_0 - R_3$) y el vector de columnas leído desde el periférico ($C_0 - C_3$). Cuando un usuario presiona una tecla, la columna respectiva es arrastrada a un nivel lógico bajo (0). El decodificador concatena ambos vectores y, de forma asíncrona, asocia dicha coordenada espacial con un código binario unívoco de 4 bits que representan caracteres del 0 al 9 y de la A a la D.

La justificación técnica de este enfoque combinacional radica en la eliminación del jitter de procesamiento y en la reducción drástica de la latencia. Al no depender de registros intermedios ni de ciclos de reloj para la traducción, el retardo del decodificador equivale únicamente al tiempo de

propagación eléctrica a través de las compuertas lógicas (LUTs) [4] de la FPGA Intel MAX 10. Esto garantiza que la identidad de la tecla presionada esté disponible de manera instantánea en el orden de los nanosegundos para ser capturada por el banco de registros, donde posteriormente, se empaquetará en las tramas cuádruples, como A001 o 0001, que el puente Arduino Uno [3] transmitirá hacia el middleware.

A continuación, se detalla la matriz de conmutación y la lógica de conversión pura de la Tabla I.

TABLA I
MATRIZ DE VERDAD COMPLETA DEL DECODIFICADOR
COMBINACIONAL. MAPEO SÍNCRONO DE LAS 16 INTERSECCIONES
FÍSICAS DEL TECLADO 4X4 A VALORES BINARIOS EQUIVALENTES DE 4
BITS.

Estado FSM	Vector Fila (R3R2R1R0)	Vector Columna (C3C2C1C0)	Tecla Física	Salida Decodificada (4 bits)
S0 FILA0	1110	1110	1	4'b0001 (1)
S0 FILA0	1110	1101	2	4'b0010 (2)
S0 FILA0	1110	1011	3	4'b0011 (3)
S0 FILA0	1110	0111	A	4'b1010 (A)
S1 FILA1	1101	1110	4	4'b0100 (4)
S1 FILA1	1101	1101	5	4'b0101 (5)
S1 FILA1	1101	1011	6	4'b0110 (6)
S1 FILA1	1101	0111	B	4'b1011 (B)
S2 FILA2	1011	1110	7	4'b0111 (7)
S2 FILA2	1011	1101	8	4'b1000 (8)
S2 FILA2	1011	1011	9	4'b1001 (9)
S2 FILA2	1011	0111	C	4'b1100 (C)
S3 FILA3	0111	1110	*	4'b1110 (E)
S3 FILA3	0111	1101	0	4'b0000 (0)
S3 FILA3	0111	1011	#	4'b1111 (F)
S3 FILA3	0111	0111	D	4'b1101 (D)

C. Banco de registros y memoria

Para mitigar la asincronía inherente a las pulsaciones del usuario y asegurar el determinismo en la conformación de instrucciones complejas, se implementó una estructura de almacenamiento intermedio a nivel de transferencia de registros (RTL) [4]. Debido a que las capas superiores de software requieren tramas formateadas de longitud fija para prevenir colisiones en el enrutamiento lógico o activaciones espurias, se justificó el diseño de un banco de registros de almacenamiento local. Esta estructura está configurada como un registro de desplazamiento de entrada y salida paralela de 32 bits de longitud total, segmentado internamente en una arquitectura jerárquica de cuatro registros síncronos contiguos de 8 bits cada uno (*Reg₃*, *Reg₂*, *Reg₁*, *Reg₀*).

D. Módulo de Transmisión Asíncrona

Una vez consolidada la palabra de 32 bits dentro de la memoria temporal, la unidad de control de hardware activa la señal de inicio de transmisión (tx_start), cediendo el flujo al módulo transceptor UART (*Universal Asynchronous Receiver-Transmitter*) emulado en el silicio de la FPGA. La función primordial de este bloque es la conversión paralelo-serie de la trama empaquetada para su posterior propagación a

través de un único canal físico unipolar hacia la capa puente. El protocolo de comunicación se parametrizó bajo un esquema estándar de un bit de inicio (*start bit* a nivel lógico bajo), ocho bits de datos lógicos, sin paridad y un bit de parada (*stop bit* a nivel lógico alto).

El mantenimiento de la tasa de transferencia crítica de 9600 baudios exige una sincronización rigurosa para prevenir errores de muestreo por desfase de fase en el receptor. La base de tiempos del bloque UART se deriva de la frecuencia del reloj maestro de 50 MHz a través de un circuito divisor secuencial basado en un contador de módulo entero. El factor de división teórico se rige por la relación matemática:

$$Divisor_{UART} = \frac{f_{CLK_MAESTRO}}{Baud\ Rate} = \frac{50 * 10^6\ Hz}{9600\ bps} = 5208.33$$

Dado que la síntesis de hardware sobre la FPGA MAX 10 se ejecuta mediante lógica discreta indexada, el módulo de transmisión adopta un divisor estático entero parametrizado con un valor de 5208. El error porcentual residual resultante es prácticamente despreciable, situándose muy por debajo del umbral crítico del 2% de tolerancia para la sincronización asíncrona. Esta precisión garantiza que los cuatro bytes de la palabra de comando se transmitan de forma fluida y sin derivas de fase temporales.

II. MIDDLEWARE E INTEGRACIÓN DE SOFTWARE

A. Middleware de interconexión

El *middleware* constituye la capa de integración lógica del sistema híbrido, permitiendo la interoperabilidad entre el comportamiento determinista del hardware y el entorno multitarea del host. Implementado en Python y diseñado para ejecutarse de forma transparente como un proceso en segundo plano, el script *puente.py* emplea la librería *pyserial* [6] para establecer una interfaz de comunicación persistente y de baja sobrecarga con el puerto COM virtual asociado al bus USB [6], configurado a una velocidad de 9600 baudios.

Su arquitectura se fundamenta en un bucle de monitorización de baja latencia respaldado por un buffer circular. El proceso permanece en espera bloqueante hasta que el subsistema de Entrada/Salida del sistema operativo detecta la llegada de datos. A continuación, el flujo recibido se decodifica bajo el estándar UTF-8 y atraviesa una etapa de sanitización destinada a eliminar caracteres de escape y otras perturbaciones transitorias. Los caracteres válidos se almacenan en el buffer y solo cuando se verifica la recepción íntegra de un paquete de 4 caracteres alfanuméricos se habilita la siguiente etapa. Finalmente, la librería *pyautogui* [7] inyecta eventos de teclado en el sistema operativo, emulando un dispositivo HID estándar y permitiendo que el navegador procese la secuencia de comandos como si proviniera de un teclado físico.

B. Arquitectura del Frontend en React

Para la visualización e interacción, se desarrolló una aplicación web basada en la biblioteca React [8], estructurada estéticamente mediante el entorno Tailwind CSS. A diferencia de las interfaces web tradicionales que requieren campos de texto explícitos para la captura de datos, esta arquitectura implementa un enfoque de escucha pasiva.

Mediante el uso del hook de ciclo de vida [8], la aplicación adjunta un listener de un evento asíncrono directamente al objeto window del navegador. Esta técnica permite interceptar de manera invisible las pulsaciones virtuales inyectadas por el middleware a nivel de sistema operativo. Los caracteres capturados son filtrados y acumulados secuencialmente en un buffer. Una vez que la estructura de datos alcanza la longitud estricta de cuatro caracteres, el sistema se detiene, la escucha temporalmente y transfiere la trama a la función que la evalúa para su enrutamiento lógico.

C. Mapeo Lógico y Gestión de Estados Reactivos

El enrutamiento de la interfaz traduce los comandos AT de cuatro dígitos en mutaciones de estado visual y funcional:

1) Control de audio y Nivelación

La recepción de los comandos 0003 y 0004 invoca una función aritmética que ajusta el estado del volumen en incrementos fijos de 6%, limitando lógicamente sus fronteras entre 0% y 100%. Por su parte, los comandos 0005 y 0006 conmutan el estado de silencio, lo cual refleja sincronamente con el estado de la barra dinámica de progreso.

2) Conmutación de Canales

Las tramas de navegación y selección directa [7] alternan el índice del canal activo en la interfaz. Esto provoca una actualización en cascada que modifica la información en la pantalla y la imagen de fondo en el panel de control.

3) Lanzamiento de plataformas

Las sentencias lógicas del 0031 al 0036 actúan como atajos de interrupción. Al procesarse, invocan una apertura de URL directa hacia servicios de streaming de terceros como Netflix, YouTube, Spotify, etc. Alineando simultáneamente con el fondo de la interfaz al contexto de la aplicación lanzada.

D. Consola de bitácora y trazabilidad

Para brindar trazabilidad temporal a la arquitectura y permitir la auditoría de los comandos provenientes del FPGA, se implementó una consola de registro histórico en la interfaz. Cada vez que una trama es decodificada y ejecutada con éxito, empaqueta un nuevo objeto de registro temporal.

En este registro también se observa un sello de tiempo en formato HH:MM:SS, generado de manera síncrona por un oscilador local del propio navegador. La matriz de registros se limita a los 10 últimos eventos mediante un truncamiento de arreglo de memoria, garantizando un rendimiento óptimo de la interfaz evitando el desbordamiento del renderizado.



Fig 4. Consola de auditoría en tiempo real, registrando la ejecución de comandos AT y su resolución de sistema.

III. PRUEBAS Y RESULTADOS EXPERIMENTALES

Para validar la eficiencia y viabilidad de la arquitectura propuesta, se sometió el sistema a una evaluación cuantitativa basada en seis métricas. Estas métricas evalúan tanto el middleware y el frontend.

A. Temporización y Latencia Global

El desempeño de un interfaz humano-máquina depende de su tiempo de respuesta. En primer lugar, se evaluó la latencia Mínima Teórica de Hardware, obteniendo un valor de 24.16ms. Este resultado se justifica matemáticamente mediante la suma del tiempo de retención del estado de la FSM el cual es 20ms y el tiempo de transmisión UART, dado que la trama consta de 4 bytes a una velocidad de 9600 baudios la transmisión se calcula de la siguiente manera:

$$T_{transmisión} = \frac{40 \text{ bits}}{9600 \text{ bps}} = 4.16 \text{ ms}$$

En segundo lugar, se midió la Latencia Global End-to-End, es decir, el tiempo transcurrido desde la pulsación del teclado hasta la verificación en el Dashboard de React [8]. El resultado arrojó un promedio de 35ms. La diferencia de 10.84ms se atribuye al proceso estocástico del sistema operativo, el cual incluye el polling del bus USB y los tiempos de despacho de kernel de Windows. A pesar de esta adición, 35ms es un valor que garantiza una muy buena experiencia de usuario.

TABLA II
COMPARACIÓN DE TIEMPOS DE RETARDO

	Tipo de retardo	Tiempo (ms)	Total
Teórico	Filtro antirrebote	20	24.16 ms
	Transmisión UART	4.16	
Real	Filtro antirrebote	20	35 ms
	Transmisión UART	4.16	
	Sistema Operativo	10.84	

B. Rendimiento Frecuencial

Para garantizar la sincronía de nuestro sistema el reloj de la FPGA fue diseñado con un tope de 49,999 ciclos de reloj sobre un reloj maestro de 50MHz, buscando un pulso de 1ms. Al evaluar el determinismo de la base de tiempos mediante el análisis del diseño RTL, se demuestra matemáticamente la generación de un Clock Enable. Esto se encuentra justificado por el hardware; a diferencia de las computadoras, la FPGA está exenta de interrupciones de sistema operativo y jitter de software, asegurando que la máquina de estados evalúe el teclado con una cadencia perfecta e invariable [4].

C. Precisión de Transmisión

La transferencia de datos hacia Python requiere una cadena estricta. La Precisión de la Tasa de Baudios UART se calculó dividiendo el reloj de 50 MHz entre los 9600 bps requeridos, resultado un factor entero de 5208. Matemáticamente este divisor genera una velocidad de transmisión real de 9600.61 bps. Esto representa un margen de error del 0.006% lo que garantiza una Tasa de Error de Bit del 0% y asegura que el buffer circular de Python reciba los caracteres sin corrupciones ni desbordamientos por desincronización.

D. Calidad de Señal y Eficiencia de Recursos

Los interruptores mecánicos, generan transitorios eléctricos indeseados que típicamente oscilan entre 5 a 12 ms [9]. Al evaluar el filtro antirrebote, la ventana de muestre digital de 20 ms demostró una gran fidelidad. Durante las pruebas de estrés, la tasa de falsos positivos fue del 0%, confirmando que ningún rebote logró fragmentar o duplicar tramas enviadas. Finalmente, el análisis de síntesis lógica mediante Quartus Prime [5] comprobó la eficiencia de Área y Potencia del diseño. La implementación del decodificador combinacional y banco de registros consumió apenas 283 Elementos Lógicos (LEs) y 84 registros dedicados, utilizando solo 40 pines.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sat Jul 4 22:26:40 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	controlador_teclado_AT
Top-level Entity Name	controlador_teclado_AT
Family	MAX 10
Device	10M50DAF484C7G
Timing Models	Final
Total logic elements	283
Total registers	84
Total pins	40
Total virtual pins	0
Total memory bits	0
Embedded Multiplier 9-bit elements	0
Total PLLs	0
UFM blocks	0
ADC blocks	0

Fig. 5. Captura de pantalla del “Compilation Report – Flow Summary” de Quartus mostrando los elementos lógicos, registros dedicados y pines utilizados.

V. DISCUSIÓN

Los resultados experimentales revelan la eficacia del particionamiento en este proyecto. El hardware se comporta de forma estrictamente determinista, aislando al sistema de las imperfecciones eléctricas y el empaquetando los datos en ráfagas exactas de 4bytes mediante su banco de registro. Esto descarga al procesador de la computadora de tareas de muestreo de bajo nivel. El cuello de botella recae, previsiblemente, en el middleware y el SO, los cuales introducen casi 11 ms de latencia. No obstante, la arquitectura basada en la escucha pasiva global en React demostró ser altamente resiliente procesando las inyecciones de comandos en tiempo real sin perder sincronía visual en el dashboard.

IV. CONCLUSIONES Y TRABAJOS FUTUROS

El presente trabajo de ingeniería concluye con el diseño, síntesis y despliegue exitoso de un controlador interactivo híbrido. La integración de un decodificador combinacional junto a un banco de memoria de desplazamiento demostró ser la arquitectura óptima para generar secuencia AT íntegras a nivel de hardware.

Las pruebas cuantitativas respaldan la robustez del modelo, el error de transmisión UART fue del 0.006%, el ruido mecánico fue suprimido en su totalidad tras obtener un 0% de falsos positivos. A nivel de software, la latencia global de 25 ms supera los requerimientos, permitiendo a la interfaz web React responder fluidamente a operaciones críticas como la nivelación de volumen, la conmutación de canales y el registro en la bitácora de historial.

Como línea de desarrollo futuro se propone reemplazar el puente UART-USB por un módulo de comunicación inalámbrica Wi-Fi. Esto permitirá a la FPGA publicar sus comandos directamente en un canal WebSockets, prescindiendo del script intermediario en Python y reduciendo aún más la latencia global del sistema.

AGRADECIMIENTO

Queremos expresar nuestro agradecimiento al profesor Hernán Villafuerte Barreto por proponernos el desarrollo de este proyecto integrador para el curso de Circuitos Digitales.

Llevar a la práctica la comunicación entre la FPGA, el microcontrolador y la interfaz web fue un reto exigente que nos ayudó a comprender realmente cómo se integran el hardware y el software. Agradecemos su guía durante las clases, así como su disposición para resolver nuestras dudas técnicas y emitimos su feedback para la mejora del proyecto.

Este trabajo nos permitió entender mucho más los conceptos teóricos, dándonos una visión clara de los problemas reales que surgen al diseñar este tipo de sistemas y cómo solucionarlos.

APÉNDICE A

CÓDIGO VERILOG

```

/*
 * MÓDULO: controlador_teclado_AT Con Transmisor
 UART
 */

module controlador_teclado_AT (
    input wire clk, // Reloj maestro
    de la FPGA (50 MHz) PIN_P11
    input wire reset, // Reset (KEY0
    activo en bajo) PIN_B8
    input wire [3:0] col, // Entradas:
    Columnas del teclado (Pull-up)
    output reg [3:0] row, // Salidas: Filas
    del teclado
    output reg led_accion, // LED físico en
    la placa (LEDRO)
    output wire [6:0] disp0, // Display HEX0
    output wire [6:0] disp1, // Display HEX1
    output wire [6:0] disp2, // Display HEX2
    output wire [6:0] disp3, // Display HEX3
    output wire tx_out // Salida Serial
    TX hacia el Arduino (Pin IO10)
);

// 1. GENERADOR DE TICK (CLOCK ENABLE): 1 pulso
cada 1 milisegundo
reg [15:0] cont_lms;
wire tick_lms = (cont_lms == 16'd49_999);
// Se hace 1 cada 50,000 ciclos

always @(posedge clk or negedge reset) begin
    if (~reset) begin
        cont_lms <= 16'd0;
    end else begin
        if (tick_lms) cont_lms <= 16'd0;
        else
            cont_lms <= cont_lms +
1'b1;
    end
end

// 2. MÁQUINA DE ESTADOS (FSM) PARA ESCANEADO DE
FILAS

localparam S0_FILA0 = 2'b00;
localparam S1_FILA1 = 2'b01;
localparam S2_FILA2 = 2'b10;
localparam S3_FILA3 = 2'b11;

reg [1:0] estado_actual, estado_siguiente;

always @(posedge clk) begin
    if (~reset) begin
        estado_actual <= S0_FILA0;
    end else if (tick_lms) begin //CLOCK ENABLE
        estado_actual <= estado_siguiente;
    end
end

always @(*) begin
    case (estado_actual)
        S0_FILA0: begin row = 4'b1110;
estado siguiente = S1_FILA1; end

```

```

        S1_FILA1: begin row = 4'b1101;
estado_siguiente = S2_FILA2; end
        S2_FILA2: begin row = 4'b1011;
estado_siguiente = S3_FILA3; end
        S3_FILA3: begin row = 4'b0111;
estado_siguiente = S0_FILA0; end
        default: begin row = 4'b1111;
estado_siguiente = S0_FILA0; end
    endcase
end

// 3. DECODIFICADOR COMBINACIONAL: MATRIZ ->
ASCII
reg [7:0] ascii_data;
reg tecla_detectada;

always @(*) begin
    ascii_data = 8'h00;
    tecla_detectada = 1'b0;
    case ({estado_actual, col})
        {S0_FILA0, 4'b1110}: begin ascii_data =
8'h31; tecla_detectada = 1'b1; end // '1'
        {S0_FILA0, 4'b1101}: begin ascii_data =
8'h32; tecla_detectada = 1'b1; end // '2'
        {S0_FILA0, 4'b1011}: begin ascii_data =
8'h33; tecla_detectada = 1'b1; end // '3'
        {S0_FILA0, 4'b0111}: begin ascii_data =
8'h41; tecla_detectada = 1'b1; end // 'A'
        {S1_FILA1, 4'b1110}: begin ascii_data =
8'h34; tecla_detectada = 1'b1; end // '4'
        {S1_FILA1, 4'b1101}: begin ascii_data =
8'h35; tecla_detectada = 1'b1; end // '5'
        {S1_FILA1, 4'b1011}: begin ascii_data =
8'h36; tecla_detectada = 1'b1; end // '6'
        {S1_FILA1, 4'b0111}: begin ascii_data =
8'h42; tecla_detectada = 1'b1; end // 'B'
        {S2_FILA2, 4'b1110}: begin ascii_data =
8'h37; tecla_detectada = 1'b1; end // '7'
        {S2_FILA2, 4'b1101}: begin ascii_data =
8'h38; tecla_detectada = 1'b1; end // '8'
        {S2_FILA2, 4'b1011}: begin ascii_data =
8'h39; tecla_detectada = 1'b1; end // '9'
        {S2_FILA2, 4'b0111}: begin ascii_data =
8'h43; tecla_detectada = 1'b1; end // 'C'
        {S3_FILA3, 4'b1110}: begin ascii_data =
8'h2A; tecla_detectada = 1'b1; end // '*'
        {S3_FILA3, 4'b1101}: begin ascii_data =
8'h30; tecla_detectada = 1'b1; end // '0'
        {S3_FILA3, 4'b1011}: begin ascii_data =
8'h23; tecla_detectada = 1'b1; end // '#'
        {S3_FILA3, 4'b0111}: begin ascii_data =
8'h44; tecla_detectada = 1'b1; end // 'D'
        default: begin ascii_data = 8'h00;
tecla_detectada = 1'b0; end
    endcase
end

// 4. GESTIÓN DE MEMORIA Y FILTRO ANTIRREBOTE
(20ms)

reg [7:0] reg0, reg1, reg2, reg3;
reg tecla_bloqueada;
reg [4:0] temporizador;
reg [1:0] ptr_direccion;

wire pulso_escritura = tecla_detectada &
~tecla_bloqueada;

// INSTANCIA DEL TRANSMISOR SERIAL (Envía la
tecla detectada a la PC)
uart_tx transmisor (
    .clk(clk),
    .tx_start(pulso_escritura),

```

```

.tx_data(ascii_data),
.tx(tx_out)
);

// Reloj 50MHz, pero la lógica avanza con el
tick_lms
always @(posedge clk) begin
    if (~reset) begin
        reg0 <= 8'h00; reg1 <= 8'h00;
        reg2 <= 8'h00; reg3 <= 8'h00;
        ptr_direccion <= 2'b00;
        tecla_bloqueada <= 1'b0;
        temporizador <= 5'd0;
    end else if (tick_lms) begin //CLOCK ENABLE

        if (tecla_detectada) begin
            tecla_bloqueada <= 1'b1;
            temporizador <= 5'd0;
        end else begin
            if (temporizador < 5'd20) begin
                temporizador <= temporizador +
1'b1;
            end else begin
                tecla_bloqueada <= 1'b0;
            end
        end

        if (pulso_escritura) begin
            case (ptr_direccion)
                2'b00: reg0 <= ascii_data;
                2'b01: reg1 <= ascii_data;
                2'b10: reg2 <= ascii_data;
                2'b11: reg3 <= ascii_data;
            endcase
            ptr_direccion <= ptr_direccion +
1'b1;
        end
    end
end

// 5. EJECUCIÓN DE COMANDO REAL (Contraseña de
4 caracteres)

always @(posedge clk) begin
    if (~reset) begin
        led_accion <= 1'b0;
    end else begin
        // Comando ON: "A001" (A, 0, 0, 1)
        if (reg0 == 8'h41 && reg1 == 8'h30 &&
reg2 == 8'h30 && reg3 == 8'h31)
            led_accion <= 1'b1;

        // Comando OFF: "A000" (A, 0, 0, 0)
        else if (reg0 == 8'h41 && reg1 == 8'h30
&& reg2 == 8'h30 && reg3 == 8'h30)
            led_accion <= 1'b0;
    end
end

// 6. DECODIFICADORES DE PANTALLA (Izquierda a
Derecha)
    decodificador_7seg u3 (.ascii(reg0),
.seg(displ3));
    decodificador_7seg u2 (.ascii(reg1),
.seg(displ2));
    decodificador_7seg u1 (.ascii(reg2),
.seg(displ1));
    decodificador_7seg u0 (.ascii(reg3),
.seg(displ0));
endmodule

// SUBMÓDULO 1: DECODIFICADOR ASCII -> 7 SEGMENTOS

```

```

module decodificador_7seg (
    input wire [7:0] ascii,
    output reg [6:0] seg
);
    always @(*) begin
        case (ascii)
            8'h30: seg = 7'b1000000; // '0'
            8'h31: seg = 7'b1111001; // '1'
            8'h32: seg = 7'b0100100; // '2'
            8'h33: seg = 7'b0110000; // '3'
            8'h34: seg = 7'b0011001; // '4'
            8'h35: seg = 7'b0010010; // '5'
            8'h36: seg = 7'b0000010; // '6'
            8'h37: seg = 7'b1111000; // '7'
            8'h38: seg = 7'b0000000; // '8'
            8'h39: seg = 7'b0011000; // '9'
            8'h41: seg = 7'b0001000; // 'A'
            8'h42: seg = 7'b0000011; // 'B'
            8'h43: seg = 7'b1000110; // 'C'
            8'h44: seg = 7'b0100001; // 'D'
            8'h2A: seg = 7'b0011100; // '*'
            8'h23: seg = 7'b1001000; // '#'
            8'h00: seg = 7'b1111111; // Apagado
            default: seg = 7'b1111111;
        endcase
    end
endmodule

// SUBMÓDULO 2: TRANSMISOR UART (9600 Baudios a 50
MHz)

module uart_tx (
    input wire clk,
    input wire tx_start,
    input wire [7:0] tx_data,
    output reg tx
);
    parameter BIT_TMR_MAX = 5208; // (50MHz / 9600
baudios)

    reg [12:0] bit_tmr = 0;
    reg [3:0] bit_idx = 0;
    reg [9:0] shift_reg = 0;
    reg estado = 0; // 0 = IDLE, 1 =
TRANSMITIENDO

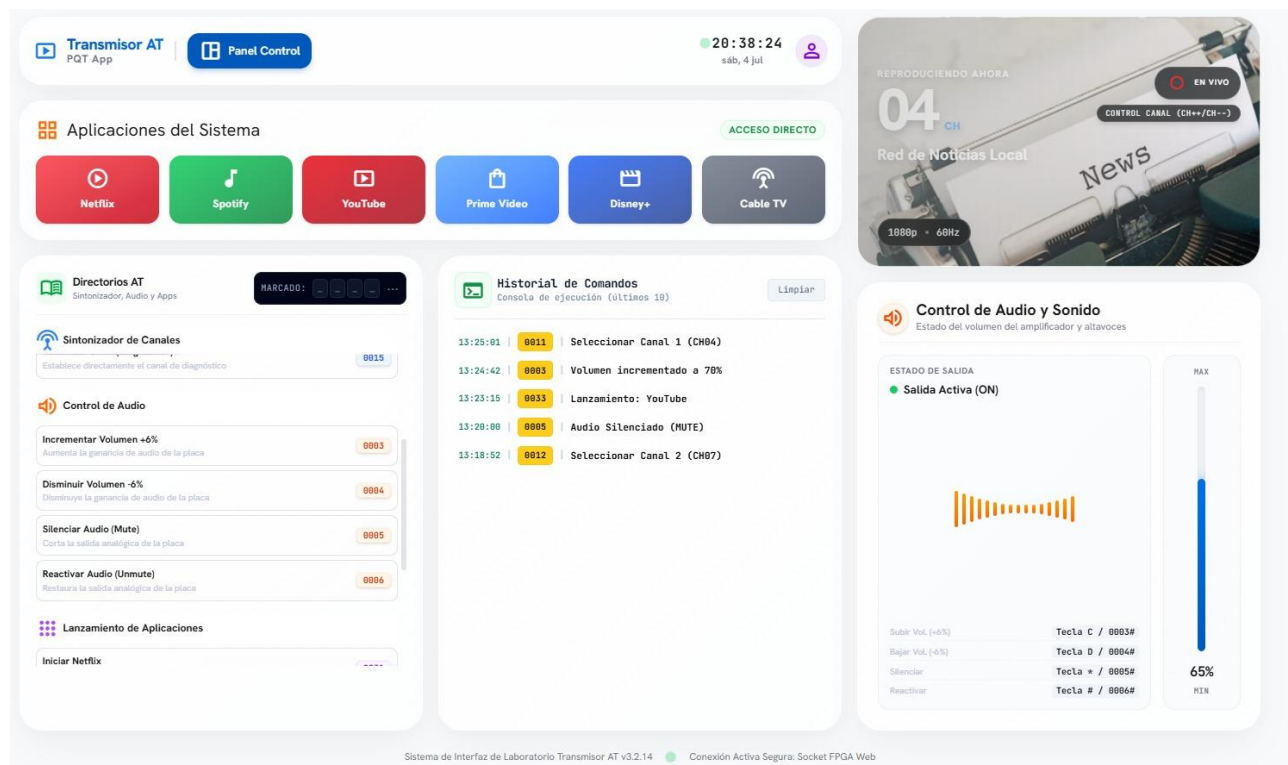
    initial tx = 1'b1;

    always @(posedge clk) begin
        if (estado == 0) begin
            tx <= 1'b1;
            if (tx_start) begin
                shift_reg <= {1'b1, tx_data, 1'b0};
// Bit Stop(1) + Datos + Bit Start(0)
                estado <= 1;
                bit_tmr <= 0;
                bit_idx <= 0;
            end
            end else begin
                if (bit_tmr == BIT_TMR_MAX - 1) begin
                    bit_tmr <= 0;
                    tx <= shift_reg[0];
                    shift_reg <= {1'b1,
shift_reg[9:1]};

                    if (bit_idx == 9) estado <= 0;
                    else bit_idx <= bit_idx + 1;
                end else begin
                    bit_tmr <= bit_tmr + 1;
                end
            end
        end
    end
endmodule

```

APÉNDICE B



REFERENCIAS

- [1] Intel Corporation, *MAX 10 FPGA Device Architecture*, Intel Altera Document 683236, 2023. [En línea]. Disponible en: <https://www.intel.com/content/www/us/en/docs/programmable/683236/>
- [2] IEEE, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2023, Mar. 2024.
- [3] Arduino, *Arduino UNO R3 - Datasheet & Hardware Documentation*, 2022. [En línea]. Disponible en: <https://docs.arduino.cc/hardware/uno-rev3>
- [4] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Cambridge, MA, USA: Morgan Kaufmann, 2021.
- [5] Intel Corporation, *Intel Quartus Prime Standard Edition User Guide: Design Compilation*, 2023. [En línea]. Disponible en: <https://www.intel.com/content/www/us/en/docs/programmable/683432/>
- [6] C. Liechti, *pySerial Documentation*, version 3.5, 2021. [En línea]. Disponible en: <https://pyserial.readthedocs.io/>
- [7] A. Sweigart, *PyAutoGUI Documentation*, 2021. [En línea]. Disponible en: <https://pyautogui.readthedocs.io/>
- [8] Meta Platforms, Inc., *React Documentation*, 2024. [En línea]. Disponible en: <https://react.dev/>
- [9] J. Ganssle, "A Guide to Debouncing," The Ganssle Group, 2004. [En línea]. Disponible en: <http://www.ganssle.com/debouncing.htm>